

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №2
з дисципліни: «Поглиблене вивчення JavaScript»
з теми: «Розробка галереї зображень із використанням API»

Виконав:
ст. гр. ПЗПІ-23-3
Білоус А. А.

Перевірив:
доц. каф. ІДС
Мар'їн С. О.

2 РОЗРОБКА ГАЛЕРЕЇ ЗОБРАЖЕНЬ ІЗ ВИКОРИСТАННЯМ API

2.1 Мета роботи

Ознайомлення з механізмами асинхронного програмування в JavaScript, опанування способів взаємодії з зовнішніми API засобами Vue 3, формування практичних навичок обробки асинхронних даних, станів завантаження та помилок у сучасних SPA-застосунках.

2.2 Хід роботи

2.2.1 Структура проєкту

Проєкт побудовано з використанням Vite + Vue 3 та складається з двох компонентів:

- «App.vue» – кореневий компонент, містить стан, логіку завантаження та шаблон сторінки;
- «components/ImageCard.vue» – компонент картки окремого зображення.

Для завантаження зображень використовується відкрите API Picsum Photos за адресою «<https://picsum.photos/v2/list?page=1&limit=20>». Відповідь є масивом об'єктів із полями «id», «author» та «download_url».

2.2.2 Змінні стану

У кореневому компоненті оголошено п'ять реактивних змінних:

```
<script setup>
import { ref, computed, onMounted } from 'vue'
import ImageCard from './components/ImageCard.vue'

const images = ref([])           // масив завантажених зображень
const isLoading = ref(false)    // стан завантаження
const error = ref(null)         // повідомлення про помилку
const favorites = ref([])       // масив id обраних зображень
const searchQuery = ref('')     // рядок пошуку
const showFavorites = ref(false) // режим відображення
</script>
```

2.2.3 Функція завантаження даних

Функція «fetchImages» виконує асинхронний HTTP-запит через «fetch». Перед запитом встановлюється прапорець «isLoading», після завершення

(незалежно від результату) він скидається у блоці «finally». При HTTP-помилці перевіряється поле «response.ok» і генерується виняток вручну.

```
async function fetchImages() {
  isLoading.value = true
  error.value = null

  try {
    const response = await fetch(
      'https://picsum.photos/v2/list?page=1&limit=20'
    )
    if (!response.ok) {
      throw new Error(`Помилка сервера: ${response.status}`)
    }
    images.value = await response.json()
  } catch (e) {
    error.value = e.message || 'Не вдалося завантажити зображення'
  } finally {
    isLoading.value = false
  }
}
```

Функція викликається у хуку «onMounted»:

```
onMounted(fetchImages)
```

2.2.4 Обчислювана властивість для фільтрації

Властивість «filteredImages» формує похідний список на основі двох параметрів: режиму відображення («showFavorites») та рядка пошуку («searchQuery»). Вона автоматично перераховується при зміні будь-якого з них.

```
const filteredImages = computed(() => {
  let list = images.value

  if (showFavorites.value) {
    list = list.filter(img => favorites.value.includes(img.id))
  }

  const query = searchQuery.value.trim().toLowerCase()
  if (query) {
    list = list.filter(img =>
      img.author.toLowerCase().includes(query))
  }

  return list
})
```

2.2.5 Функціонал обраних зображень

Обрані зберігаються локально у масиві ідентифікаторів. Функція «toggleFavorite» перевіряє наявність «id» у масиві й або додає, або видаляє його:

```
function toggleFavorite(id) {
  const index = favorites.value.indexOf(id)
  if (index === -1) {
    favorites.value.push(id)
  } else {
    favorites.value.splice(index, 1)
  }
}
```

2.2.6 Шаблон кореневого компонента

Шаблон побудований за принципом умовного відображення: залежно від стану показується індикатор завантаження, повідомлення про помилку або сітка карток.

```
<template>
  <div class="container">
    <div class="toolbar">
      <input v-model="searchQuery" type="text"
        placeholder="Пошук за автором" />
      <div class="view-toggle">
        <button :class="{ active: !showFavorites }"
          @click="showFavorites = false">Всі</button>
        <button :class="{ active: showFavorites }"
          @click="showFavorites = true">Обрані</button>
      </div>
    </div>

    <div v-if="isLoading">Завантаження...</div>

    <div v-else-if="error">
      <p>{{ error }}</p>
      <button @click="fetchImages">Спробувати ще раз</button>
    </div>

    <div v-else-if="filteredImages.length === 0">Нічого не знайдено</div>

    <div v-else class="gallery">
      <ImageCard
        v-for="image in filteredImages"
        :key="image.id"
        :image="image"
        :isFavorite="favorites.includes(image.id)"
        @toggleFavorite="toggleFavorite"
      />
    </div>
  </div>
```

```

</div>
</template>

```

2.2.7 Компонент ImageCard.vue

Компонент відображає картку зображення: фото (завантажується через «<https://picsum.photos/id/{id}/400/300>»), ім'я автора, ідентифікатор та кнопку обраного. Активний стан кнопки відображається через умовне додавання класу «:class="{ active: isFavorite }"».

```

<script setup>
defineProps({
  image: { type: Object, required: true },
  isFavorite: { type: Boolean, default: false },
})
const emit = defineEmits(['toggleFavorite'])
</script>

<template>
  <div class="card">
    <div class="card-img-wrap">
      
    </div>
    <div class="card-footer">
      <div class="card-info">
        <span class="author">{{ image.author }}</span>
        <span class="image-id">#{{ image.id }}</span>
      </div>
      <button class="fav-btn" :class="{ active: isFavorite }"
        @click="emit('toggleFavorite', image.id)">★</button>
    </div>
  </div>
</template>

```

2.3 Висновки

У ході лабораторної роботи розроблено галерею зображень із використанням зовнішнього API Picsum Photos. Реалізовано асинхронне завантаження даних через «fetch» з обробкою стану завантаження та помилок. Застосовано хук «onMounted» для ініціювання запиту після монтування компонента. Фільтрацію та пошук реалізовано через обчислювану властивість «computed», яка автоматично реагує на зміни стану. Функціонал обраних збережено локально у реактивному масиві ідентифікаторів. Застосунок розподілено на два компоненти: кореневий «App.vue» та «ImageCard.vue».

2.4 Відповіді на контрольні запитання

- а) **Що таке асинхронне програмування і чому воно необхідне у вебзастосунках?** Асинхронне програмування – підхід, при якому тривалі операції (мережеві запити, читання файлів) виконуються у фоновому режимі, не блокуючи головний потік виконання. У вебзастосунках це необхідно, щоб браузер залишався чуйним під час очікування відповіді від сервера: сторінка не зависає, а користувач може взаємодіяти з інтерфейсом.
- б) **Яке призначення ключових слів `async` та `await`?** Ключове слово «`async`» перетворює функцію на асинхронну – вона завжди повертає «`Promise`». Всередині такої функції можна використовувати «`await`» перед будь-яким «`Promise`»: виконання функції призупиняється до отримання результату, але головний потік при цьому не блокується. Разом вони дозволяють писати асинхронний код у лінійному стилі без вкладених колбеків.
- в) **Як виконати HTTP-запит за допомогою `fetch`? Яку структуру має відповідь?** Запит виконується викликом «`fetch(url)`», який повертає «`Promise<Response>`». Об'єкт «`Response`» містить поля «`ok`» (true якщо статус 200–299), «`status`» (числовий код), та методи «`json()`», «`text()`» для читання тіла відповіді. Оскільки «`fetch`» не кидає виняток при кодах 4xx/5xx, успішність слід перевіряти через «`response.ok`».
- г) **Навіщо використовується хук `onMounted`? На якому етапі він виконується?** «`onMounted`» виконується після того, як компонент повністю вставлено у DOM і всі дочірні компоненти також змонтовано. Це гарантує, що DOM-елементи вже існують і користувач бачить інтерфейс. Саме тому тут рекомендується запускати завантаження початкових даних – запит виконується одразу, але сторінка вже відрендерена і може показати індикатор завантаження.
- д) **Що таке стан завантаження (`loading state`) і як його реалізувати у `Vue`?** Стан завантаження – це реактивна булева змінна («`isLoading`»), яка сигналізує застосунку, що запит ще виконується. У `Vue` вона

оголошується через «`ref(false)`», встановлюється у «`true`» перед запитом і скидається у «`false`» у блоці «`finally`». У шаблоні директива «`v-if="isLoading"`» умовно показує індикатор.

- е) **Як обробити помилку мережевого запиту за допомогою `try/catch`?** Конструкція «`try/catch`» обгортає асинхронний код: якщо «`fetch`» або «`response.json()`» кидають виняток (мережева помилка, некоректний JSON), він перехоплюється у блоці «`catch`». Там зберігається повідомлення в реактивну змінну «`error`». Блок «`finally`» виконується завжди – у ньому скидається прапорець завантаження.
- ж) **Чим `computed` відрізняється від звичайної функції у Vue?** «`computed`» – це кешована обчислювана властивість: вона перераховується лише тоді, коли змінюється одна з її реактивних залежностей. Звичайна функція виконується заново при кожному зверненні. Для похідних даних (фільтрація, сортування) «`computed`» є ефективнішим вибором, оскільки уникає зайвих обчислень при ререндері.
- и) **Як реалізувати локальний список обраних без збереження на сервері?** Достатньо оголосити реактивний масив ідентифікаторів: «`const favorites = ref([])`». При кліку на кнопку перевіряється наявність «`id`» у масиві: якщо немає – додається («`push`»), якщо є – видаляється («`splice`»). Для визначення стану кнопки використовується «`favorites.value.includes(id)`». Такий підхід зберігає дані лише у пам'яті браузера поточної сесії.